



Complete Field Notes for Junior Data Scientists

From Zero to Production — with Advanced Patterns

7	Levels	50+	Code Snippets
Best	Practices	Advanced	Patterns

requests · BeautifulSoup · Selenium · Playwright
Pagination · Sessions & Headers · JSON APIs
Production-grade Scrapers · Retry Logic · Data Cleaning
Ethical Scraping · Anti-bot Evasion · Async Patterns

■ TABLE OF CONTENTS

Level 0	How Websites Work — HTML Primer
Level 1	Downloading Pages with requests
Level 2	Parsing HTML with BeautifulSoup
Level 3	Pagination — Scraping Multiple Pages
Level 4	Headers & Sessions — Look Like a Browser
Level 5	Scraping JSON APIs
Level 6	Dynamic Pages with Selenium
Level 7	Production-Grade Scraper
Advanced	Async Scraping with asyncio + aiohttp
Advanced	XPath & CSS Selector Deep Dive
Advanced	Middleware, Proxies & Rate-Limit Handling
Reference	Quick Reference Card
Ethics	Legal & Ethical Rules

Before writing a single line of scraping code, you need to understand what actually happens when your browser loads a page — because scraping is just replicating those steps in Python.

The Request-Response Cycle

- 1. REQUEST** Your browser (or Python) sends an HTTP GET to a URL.
- 2. RESPONSE** The server replies with HTML text — the raw source of the page.
- 3. RENDER** Your browser turns that HTML into pixels. Python skips this step.

HTML Crash Course — Tags You'll See Everywhere

Every visible element on a page lives inside an HTML *tag*. Tags have a name, optional attributes (like class or id), and content.

```
<h1>Big Title</h1> # heading – main title of a section

<p>Paragraph text here.</p> # paragraph – body text

<a href="https://...">Link</a> # anchor tag – a clickable link

<div class="price">$29.99</div> # div – generic container, very common

<span id="qty">3</span> # span – inline container

 # image (self-closing)

<table><tr><td>Cell</td></tr></table> # table > row > cell

<ul><li>Item one</li></ul> # unordered list > list item
```

Common HTML tags — learn these and you can scrape 90% of pages

■ TIP

In Chrome or Firefox: right-click any text on a page → Inspect. The DevTools panel highlights the exact HTML tag. Copy its class or id into your BeautifulSoup selector.

CSS Classes and IDs — How to Target Elements

```
<!-- class → many elements can share it (use find_all) -->

<div class="product-card">...</div>
```

```
<div class="product-card">...</div>

<!-- id → unique per page (use find) -->

<div id="main-header">...</div>

<!-- Multiple classes on one element -->

<p class="price sale featured">$9.99</p>
```

class vs id in HTML

The **requests** library is the simplest way to send HTTP requests from Python. Think of it as a browser without a screen — it fetches the raw HTML for you.

Installation

```
pip install requests beautifulsoup4 lxml pandas
```

Run once in your terminal

Basic GET Request

```
import requests

url = 'https://books.toscrape.com'

response = requests.get(url)

# Check if the request succeeded

print(response.status_code) # 200 = OK

print(response.text[:300]) # First 300 chars of HTML
```

HTTP Status Codes — What They Mean

Code	Meaning	What to do
200	OK — success	Proceed with parsing
301/302	Redirect	requests follows it automatically
403	Forbidden	Site is blocking you; try adding headers
404	Not Found	Wrong URL; check your link
429	Too Many Requests	You're being rate-limited; slow down
500	Server Error	Site issue; retry later

■■ NOTE

Always check **response.status_code == 200** before parsing. Parsing a 403 or 404 page will give you garbage data silently.

Passing Query Parameters

```
# Instead of building URLs like: ?page=2&sort=price

# Let requests build them for you:

params = {

    'page': 2,

    'sort': 'price',

    'category': 'books',

}

response = requests.get('https://example.com/search', params=params)

print(response.url) # Shows the full URL that was sent
```

params= is cleaner and safer than string concatenation

BeautifulSoup turns a raw HTML string into a tree of objects you can search and navigate. It's the core tool for extracting structured data from pages.

Core Methods

Method	Returns	Use when
<code>soup.find('tag')</code>	First match or None	You expect one result (title, nav)
<code>soup.find('tag', class_='x')</code>	First with that class	Targeting a specific CSS class
<code>soup.find_all('tag')</code>	List of all matches	Collecting many items (products, rows)
<code>soup.select('css selector')</code>	List (CSS selector)	Complex targeting (div > ul > li.active)
<code>element.text</code>	String	Get visible text inside a tag
<code>element.get_text(strip=True)</code>	Cleaned string	Strip whitespace automatically
<code>element['href']</code>	Attribute value	Get link URLs, src, class values

Practical Example — Scraping a Book Listing

```
import requests

from bs4 import BeautifulSoup

response = requests.get('https://books.toscrape.com')

soup = BeautifulSoup(response.text, 'lxml')

# Each book lives inside <article class='product_pod'>

books = soup.find_all('article', class_='product_pod')

print(f'Found {len(books)} books') # → 20

for book in books[:3]:

    # Title is in <h3><a title='...'> – note we grab the attribute, not .text

    title = book.find('h3').find('a')['title']

    # Price text includes a currency symbol: '£12.99'
```

```
price = book.find('p', class_='price_color').text.strip()

# Rating is encoded as a CSS class name: 'One', 'Two', etc.

rating = book.find('p', class_='star-rating')['class'][1]

print(f'{title[:35]:<35} {price} ★ {rating}')
```

Working example on books.toscrape.com — safe practice site

Navigating the Tag Tree

```
# Moving DOWN the tree

soup.find('div').find('ul').find('li') # chained search

soup.find('div').find_all('a') # all links inside a div


# Moving UP the tree

element.parent # the tag containing this element

element.parents # generator of all ancestors


# Moving SIDEWAYS

element.next_sibling # next tag at the same level

element.previous_sibling


# CSS selectors (very powerful)

soup.select('div.product-card > h2') # direct child

soup.select('table tr:nth-of-type(2)') # second row

soup.select('[data-price]') # has attribute
```

■■ NOTE

When scraping fails silently (you get None everywhere), open the page in your browser, view source, and search for the exact class name. Websites sometimes use slightly different class names on different pages.

Real data spans many pages. The key pattern: find the 'Next' button on each page, extract its URL, and loop until there's no next page.

```
import requests, time, random

from bs4 import BeautifulSoup

import pandas as pd

def scrape_all_books(max_pages=5):

    base = 'https://books.toscrape.com/catalogue/'

    url = base + 'page-1.html'

    books = []

    page = 1

    while url and page <= max_pages:

        print(f'Page {page}: {url}')

        soup = BeautifulSoup(requests.get(url).text, 'lxml')

        for article in soup.find_all('article', class_='product_pod'):

            books.append({

                'title': article.find('h3').find('a')['title'],

                'price': article.find('p', class_='price_color').text.strip(),

                'rating': article.find('p', class_='star-rating')['class'][1],

            })

        # Detect 'Next' button – if missing, we're on the last page

        nxt = soup.find('li', class_='next')

        url = base + nxt.find('a')['href'] if nxt else None

        page += 1
```

```

time.sleep(random.uniform(1, 3)) # be polite!

return pd.DataFrame(books)

df = scrape_all_books(max_pages=3)

print(df.shape) # (60, 3) — 20 books × 3 pages

```

Pagination loop — the backbone of most scrapers

Data Cleaning After Scraping

```

# Convert '£12.99' → 12.99 (float)

df['price_num'] = (df['price']

.str.replace('£', '', regex=False)

.str.replace('Â', '', regex=False) # encoding artifact

.astype(float))

# Map rating words to numbers

rating_map = {'One':1, 'Two':2, 'Three':3, 'Four':4, 'Five':5}

df['rating_num'] = df['rating'].map(rating_map)

# Quick analysis

print(df.groupby('rating_num')['price_num'].mean().round(2))

# Save

df.to_csv('books.csv', index=False) # CSV

df.to_json('books.json', orient='records', indent=2) # JSON

```

By default, requests announces itself as 'python-requests'. Many sites block this immediately. Sending real browser headers — and using a Session for cookies — is essential for production scraping.

Realistic Browser Headers

```
HEADERS = {

    'User-Agent': (

        'Mozilla/5.0 (Windows NT 10.0; Win64; x64) '

        'AppleWebKit/537.36 (KHTML, like Gecko) '

        'Chrome/124.0.0.0 Safari/537.36'

    ),

    'Accept-Language': 'en-US,en;q=0.9',

    'Accept': 'text/html,application/xhtml+xml;q=0.9,*/*;q=0.8',

    'Referer': 'https://www.google.com/', # pretend you came from Google

    'DNT': '1', # Do Not Track

    'Connection': 'keep-alive',

}
```

Copy-paste these headers for most sites that require them

Using a Session

A Session object persists cookies and headers across all requests. Essential for sites with logins, or any multi-page scrape.

```
session = requests.Session()

session.headers.update(HEADERS)

# Session keeps cookies automatically between requests

response = session.get('https://books.toscrape.com')

# Logging in to a site (form-based login)

session.post('https://example.com/login', data={
```

```
'username': 'myuser',  
  
'password': 'mypassword',  
  
})  
  
# Now all subsequent requests are authenticated  
  
private = session.get('https://example.com/dashboard')
```

■ TIP

Rotate your User-Agent string between requests when scraping many pages from the same site. The **fake-useragent** library (pip install fake-useragent) generates realistic random agents automatically.

Modern single-page apps load data via hidden API calls (XHR/Fetch requests). These are MUCH easier to scrape than HTML — the data is already structured. Finding them is often the hardest part.

How to Find Hidden APIs

1. Open DevTools (F12 in Chrome/Firefox)
2. Go to the Network tab
3. Reload the page or click around to trigger data loading
4. Click 'Fetch/XHR' filter to hide CSS/images
5. Look for requests returning JSON — click them, check the Response tab
6. Copy that URL directly into Python

```
import requests

# Example: Open-Meteo free weather API (no key required)

def get_weather(lat, lon):

    url = 'https://api.open-meteo.com/v1/forecast'

    params = {

        'latitude': lat,

        'longitude': lon,

        'current': 'temperature_2m,wind_speed_10m',

        'timezone': 'auto',

    }

    r = requests.get(url, params=params)

    r.raise_for_status() # throw exception on error

    return r.json() # auto-parses JSON → Python dict

data = get_weather(40.71, -74.01) # New York

print(data['current']['temperature_2m'], '°C')

print(data['current']['wind_speed_10m'], 'km/h')
```

JSON API — cleaner and faster than HTML parsing

Handling Paginated APIs

```
def get_all_pages(base_url, params):  
  
    results = []  
  
    page = 1  
  
    while True:  
  
        params['page'] = page  
  
        data = requests.get(base_url, params=params).json()  
  
        # Stop conditions vary by API:  
  
        if not data.get('results'): # empty results  
  
            break  
  
        if page >= data.get('total_pages', 1): # reached the end  
  
            break  
  
        results.extend(data['results'])  
  
        page += 1  
  
        time.sleep(0.5)  
  
    return results
```

JavaScript-heavy pages render their content in the browser after the initial HTML loads. requests cannot see this content — you need a real browser. Selenium (or Playwright) drives Chrome/Firefox from Python.

Installation

```
pip install selenium

# Then download ChromeDriver matching your Chrome version:

# https://chromedriver.chromium.org/downloads

# OR use the new automatic driver management:

pip install selenium webdriver-manager
```

Core Selenium Pattern

```
from selenium import webdriver

from selenium.webdriver.common.by import By

from selenium.webdriver.support.ui import WebDriverWait

from selenium.webdriver.support import expected_conditions as EC

from selenium.webdriver.chrome.options import Options

options = Options()

options.add_argument('--headless') # no visible window

options.add_argument('--no-sandbox') # required on Linux

options.add_argument('--disable-dev-shm-usage')

driver = webdriver.Chrome(options=options)

try:

    driver.get('https://books.toscrape.com')

    # ALWAYS wait for the element — JS pages load slowly
```

```

wait = WebDriverWait(driver, 10)

wait.until(EC.presence_of_element_located(
    (By.CSS_SELECTOR, 'article.product_pod')
))

books = driver.find_elements(By.CSS_SELECTOR, 'article.product_pod')

for b in books[:3]:
    title = b.find_element(By.CSS_SELECTOR, 'h3 a').get_attribute('title')
    price = b.find_element(By.CSS_SELECTOR, '.price_color').text
    print(title, price)

# Interact with the page

driver.find_element(By.ID, 'search').send_keys('python')

driver.find_element(By.ID, 'search-btn').click()

finally:

    driver.quit() # ALWAYS close — even on error

```

Selenium skeleton — copy this as a starting template

■ TIP

Playwright is a modern alternative to Selenium. It's faster, more reliable, and has a great async API. Install: **`pip install playwright && playwright install chromium`**

A scraper you run once manually is very different from one running daily in a data pipeline. Production scrapers need retry logic, structured logging, data validation, and clean separation of concerns.

Key Production Patterns

Retry with backoff — Retry failed requests 3×, waiting 2s, 4s, 8s between tries.

Structured logging — Log every page, timing, and error to a file with timestamps.

Data validation — Reject records with missing/invalid fields before saving.

Class-based design — Group fetch, parse, and save into a single class for testability.

Random delays — `time.sleep(random.uniform(1.0, 3.0))` between every request.

Dataclasses — Use `@dataclass` for typed, structured data containers.

```
import requests, time, random, logging

from dataclasses import dataclass

from bs4 import BeautifulSoup

import pandas as pd

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[logging.StreamHandler(), logging.FileHandler('scraper.log')]
)

log = logging.getLogger(__name__)

@dataclass
class Book:
    title: str
    price: float
    rating: int
    url: str

    def is_valid(self) -> bool:
```

```

return bool(self.title) and self.price > 0 and 1 <= self.rating <= 5

class BookScraper:

    BASE = 'https://books.toscrape.com/catalogue/'

    START = BASE + 'page-1.html'

    RATINGS = {'One':1, 'Two':2, 'Three':3, 'Four':4, 'Five':5}

    def __init__(self):

        self.session = requests.Session()

    def _fetch(self, url): # retry logic

        for attempt in range(1, 4):

            try:

                r = self.session.get(url, timeout=10)

                r.raise_for_status()

                return BeautifulSoup(r.text, 'lxml')

            except requests.RequestException as e:

                log.warning('Attempt %d failed: %s', attempt, e)

                time.sleep(2 ** attempt)

        return None # all retries failed

    def _parse(self, article): # parse one book

        try:

            price = float(article.find('p', class_='price_color')

                .text.strip().replace('£', '').replace('Â', ''))

            return Book(

                title = article.find('h3').find('a')['title'],

                price = price,

                rating = self.RATINGS[article.find('p',

                    class_='star-rating')['class'][1]],

```

```

url = self.BASE + article.find('h3').find('a')['href'],
)

except Exception as e:

log.warning('Parse error: %s', e)

return None


def run(self, max_pages=5) -> pd.DataFrame:

books, url, page = [], self.START, 1

while url and page <= max_pages:

log.info('Page %d', page)

soup = self._fetch(url)

if soup is None: break

for art in soup.find_all('article', class_='product_pod'):

b = self._parse(art)

if b and b.is_valid(): books.append(b)

nxt = soup.find('li', class_='next')

url = self.BASE + nxt.find('a')['href'] if nxt else None

page += 1

time.sleep(random.uniform(1.0, 3.0))

log.info('Done. %d books collected.', len(books))

return pd.DataFrame([b.__dict__ for b in books])

```

Complete production scraper template — adapt for any site

■ ADVANCED — Async Scraping with asyncio + aiohttp

Standard requests is *synchronous* — it waits for each page to download before starting the next. With asyncio and aiohttp you can fetch dozens of pages in parallel, dramatically cutting runtime.

Sync vs Async — The Key Difference

```
# SYNC: total time = sum of all page times

for url in urls: # waits for each one

    response = requests.get(url)


# ASYNC: total time ≈ slowest single page

async def fetch(session, url):

    async with session.get(url) as response:

        return await response.text()


# All fetches start almost simultaneously

results = await asyncio.gather(*[fetch(s, u) for u in urls])
```

Full Async Scraper

```
import asyncio

import aiohttp # pip install aiohttp

from bs4 import BeautifulSoup

SEMAPHORE = asyncio.Semaphore(5) # max 5 concurrent requests


async def fetch_page(session, url):

    async with SEMAPHORE: # respect concurrency limit

        async with session.get(url, timeout=aiohttp.ClientTimeout(total=10)) as r:

            return await r.text()


async def parse_books(html):

    soup = BeautifulSoup(html, 'lxml')

    return [
```

```

{
    'title': a.find('h3').find('a')['title'],
    'price': a.find('p', class_='price_color').text.strip(),
}

for a in soup.find_all('article', class_='product_pod')
]

async def scrape_async(page_urls):
    async with aiohttp.ClientSession() as session:
        # Fetch all pages concurrently
        html_pages = await asyncio.gather(*[
            fetch_page(session, url) for url in page_urls
        ])
        # Parse all pages
        all_books = []
        for html in html_pages:
            all_books.extend(await parse_books(html))
        return all_books

# Build list of page URLs
urls = [f'https://books.toscrape.com/catalogue/page-{i}.html'
        for i in range(1, 11)]

books = asyncio.run(scrape_async(urls))
print(f'Got {len(books)} books')

```

Async scraper — ~10x faster for I/O-bound workloads

■■ NOTE

Always use a **Semaphore** to cap concurrent requests. Launching 50 simultaneous requests will likely get you banned and could harm the server. 5–10 concurrent is usually safe.

■ ADVANCED — XPath & CSS Selectors Deep Dive

CSS selectors and XPath are the two targeting languages for HTML. Mastering them lets you extract any element precisely, even without obvious class names or IDs.

CSS Selectors — Cheat Sheet

Selector	Meaning	Example
tag	All tags of type	soup.select('a')
.class	Elements with class	soup.select('.price')
#id	Element with id	soup.select('#main')
A > B	B direct child of A	soup.select('ul > li')
A B	B anywhere inside A	soup.select('div a')
A + B	B immediately after A	soup.select('h2 + p')
[attr]	Has attribute	soup.select('[href]')
[attr=val]	Attribute equals value	soup.select('[type=text]')
[attr*=val]	Attribute contains value	soup.select('[class*=price]')
:first-child	First child element	soup.select('li:first-child')
:nth-of-type(2)	Second of its type	soup.select('tr:nth-of-type(2)')

XPath — For lxml and Selenium

[illegible]

```
driver.find_element(By.XPATH, '//button[text()="Add to Cart"]')  
  
driver.find_elements(By.XPATH, '//div[@data-product-id]')
```

When scraping at scale, you will hit IP bans, CAPTCHAs, and rate limits. This section covers the standard defences.

Rotating Proxies

```
import random

PROXIES = [

    'http://user:pass@proxy1.example.com:8080',

    'http://user:pass@proxy2.example.com:8080',

    'http://user:pass@proxy3.example.com:8080',

]

def get_with_proxy(url):

    proxy = random.choice(PROXIES)

    return requests.get(url, proxies={'http': proxy, 'https': proxy},

        timeout=10)

# Free proxy services (unreliable): proxy-list.download

# Paid services (reliable): BrightData, Oxylabs, ScraperAPI
```

Exponential Backoff on Rate Limits

```
import time

from requests.adapters import HTTPAdapter

from urllib3.util.retry import Retry

# Auto-retry on 429 (rate limit) and 500 (server error)

retry_strategy = Retry(

    total=5,

    status_forcelist=[429, 500, 502, 503, 504],

    backoff_factor=2, # wait 2, 4, 8, 16, 32 seconds
```



```

respect_retry_after_header=True, # honour server's Retry-After
)

adapter = HTTPAdapter(max_retries=retry_strategy)

session = requests.Session()

session.mount('http://', adapter)

session.mount('https://', adapter)

# Now this will auto-retry on failures:

response = session.get('https://example.com/api/data')

```

Mount this adapter on your session — handles most transient failures

Rotating User Agents

```

from fake_useragent import UserAgent # pip install fake-useragent

ua = UserAgent()

for url in url_list:

    headers = {'User-Agent': ua.random} # different agent each time

    response = requests.get(url, headers=headers)

    time.sleep(random.uniform(1, 4))

```

■ TIP

The single most effective anti-detection technique is adding realistic, variable delays between requests. A bot that waits 1.7s, then 2.3s, then 1.1s looks far more human than one that always waits exactly 1.0s.

■ QUICK REFERENCE CARD

Task	Code
Download a page	<code>requests.get(url)</code>
Parse HTML	<code>BeautifulSoup(html, 'lxml')</code>
Find first tag	<code>soup.find('div', class_='x')</code>
Find all tags	<code>soup.find_all('a')</code>
CSS selector	<code>soup.select('div > ul > li.active')</code>
Get text	<code>element.text.strip()</code>
Get attribute	<code>element['href']</code>
Check status	<code>response.status_code == 200</code>
Parse JSON response	<code>response.json()</code>
Add browser headers	<code>session.headers.update(HEADERS)</code>
Wait between requests	<code>time.sleep(random.uniform(1, 3))</code>
Save to CSV	<code>df.to_csv('out.csv', index=False)</code>
Save to JSON	<code>df.to_json('out.json', orient='records')</code>
Async fetch	<code>await session.get(url)</code>
Selenium wait	<code>WebDriverWait(driver, 10).until(EC.presence_of_element_located(...))</code>
Retry adapter	<code>HTTPAdapter(max_retries=Retry(total=5))</code>

■■ ETHICS & LEGAL RULES

■ DO

- Check robots.txt first → <https://example.com/robots.txt>
- Add delays between requests (1–3 seconds minimum)
- Read the site's Terms of Service before scraping
- Only scrape public data — never bypass authentication
- Cache pages locally to avoid redundant requests

■ DON'T

- Scrape personal/private data without consent (GDPR liability)
- Bypass CAPTCHAs or authentication systems
- Scrape at high speed — you can take down small sites
- Use scraped data commercially without permission
- Ignore a Cease & Desist from a website owner

■ PRACTICE SITES & NEXT STEPS

books.toscrape.com — Book store — made for scraping learners
quotes.toscrape.com — Quotes site — multiple scraping challenges
httpbin.org — Echo server — test your requests and headers
open-meteo.com — Free weather API — no key needed
playwright.dev/python — Modern Selenium alternative — async-native
scrapy.org — Full scraping framework for large-scale jobs